

Rajarata University of Sri Lanka

Faculty of Technology

Department of ICT



uniScholar

Smart Resource Sharing Platform for Students

Final Project Report

ICT 1209 – Web Technologies | Mini Project | 2023/24 Batch

Project Title	uniScholar – Smart Resource Sharing Platform for Students
Theme	Student Resource Hub
Group Number	35
Group Members	A.K. Nisal Nethsara (ITT/2024/072) A.M.I.S. Silva (ITT/2024/103)
Course	ICT 1209 – Web Technologies
Submission	Phase 3 – Final Report
Date	September 2026

Contents

1. Project Overview	3
1.1 Purpose and Goals	3
1.2 Target Audience	3
1.3 Key Features Implemented	3
2. System Design	5
2.1 Software Architecture	5
2.2 Database Schema	5
2.3 Folder Structure	7
3. Implementation	8
3.1 User Authentication	8
3.2 Role-Based Dashboards	9
3.3 Classroom Creation and Joining	10
3.4 Notice Board and File Download	10
3.5 Live Search	11
3.6 GPA Calculator (Client-Side JavaScript)	11
3.7 Responsive Design and Visual Interest	11
3.8 Database Evidence	11
4. Challenges and Solutions	13
Challenge 1: Missing student_classrooms table	13
Challenge 2: Column mismatch when saving a classroom	13
Challenge 3: Contact form not connected to the database	13
Challenge 4: Some pages are placeholders	14
Challenge 5: Learning PHP sessions and prepared statements	14
5. Individual Contribution	15
6. Setup Instructions	16
Step 1: Install requirements	16

Step 2: Get the project files	16
Step 3: Start the server	16
Step 4: Set up the database	16
Step 5: Check the connection settings	16
Step 6: Open the application.....	16
7. References	17

1. Project Overview

uniScholar is a web application built for the ICT 1209 – Web Technologies mini project. It works like a shared study hub for university students. Students often keep their notes, past papers, and class updates in many different places, like phone galleries, chat groups, or random cloud folders. This makes it hard to find things later. uniScholar tries to fix this problem by giving every class one shared place for its files and updates.

The system has two types of users: a Student and an Administrator. A student can register, log in, join a classroom using a course code and an enrollment key, and then view notices and download files that the admin has shared for that class. An administrator can log in to a separate dashboard, create new classrooms, and post updates with attached files.

1.1 Purpose and Goals

- Give students one shared place to find class notes, past papers, and notices.
- Let an admin manage classrooms and post updates without needing technical help.
- Practice building a full web application with PHP and MySQL, from login to file sharing.
- Show good use of HTML5, CSS3, Bootstrap 5, vanilla JavaScript, PHP 8, and MySQL, as required by the module.

1.2 Target Audience

The platform is aimed at undergraduate students and academic staff of the Faculty of Technology, Rajarata University of Sri Lanka, across departments such as ICT, Food Technology, Bioprocess Technology, Materials Technology, and Electrical & Electronics Technology. Any student can register, join classrooms for their own subjects, and share or receive resources with classmates.

1.3 Key Features Implemented

- **Secure Authentication:** Registration and login with hashed passwords (PHP password_hash / password_verify) and PHP sessions.
- **Role-Based Dashboards:** Separate dashboard views and menus for students and admins, based on the role stored in the database.
- **Classroom Creation (Admin):** Admins can create a classroom with a name, course code, semester, study year, faculty, and a secret enrollment key.
- **Classroom Join (Student):** Students join a classroom by entering the course code and the enrollment key given by the admin.
- **Notice Board & File Sharing:** Admins post updates (title, description, and an optional attached file) to a classroom. Students see these updates and can download the attached file.

- **Live Search:** A JavaScript-powered search box that filters page links as the student types, without reloading the page.
- **GPA Calculator:** A client-side JavaScript tool where a student can add course grades and instantly see their GPA, with no page reload needed.
- **Responsive UI:** Built with Bootstrap 5 so pages adjust to desktop, tablet, and mobile screens.

Some features from the original proposal, such as the contact form and the past-paper archive, were started but are not fully connected to the database yet. These are explained honestly in Section 4 (Challenges and Solutions) together with what would be needed to finish them.

2. System Design

2.1 Software Architecture

uniScholar uses a simple three-layer (client-server) design, which is the normal setup for a PHP and MySQL project run on XAMPP:

- **Client Layer:** The browser shows pages built with HTML5, CSS3, and Bootstrap 5, and runs small JavaScript scripts for search, the GPA calculator, and the image slider.
- **Server Layer:** An Apache server runs PHP 8 scripts that read form input, check sessions, and talk to the database using prepared statements (mysqli).
- **Data Layer:** A MySQL database named unischolar stores users, classrooms, and updates. Uploaded files are saved on the server's file system and linked to database rows by their file path.



Figure 1: uniScholar three-layer architecture

2.2 Database Schema

The database is created automatically the first time the app runs, inside database/connection.php. The main tables are:

- **users** – stores each account: name, email, hashed password, role (user or admin), university, faculty, study year, and semester.
- **classrooms** – stores each class created by an admin: classroom name, course code, enrollment key, semester, study year, faculty, and status.
- **classroom_updates** – stores each notice/post made inside a classroom, with an optional file attachment. Linked to classrooms by classroom_id.
- **files** – a general table for storing uploaded file records (name, path, upload time).
- **student_classrooms** – used by the code to record which student has joined which classroom (by student_id and course_code).

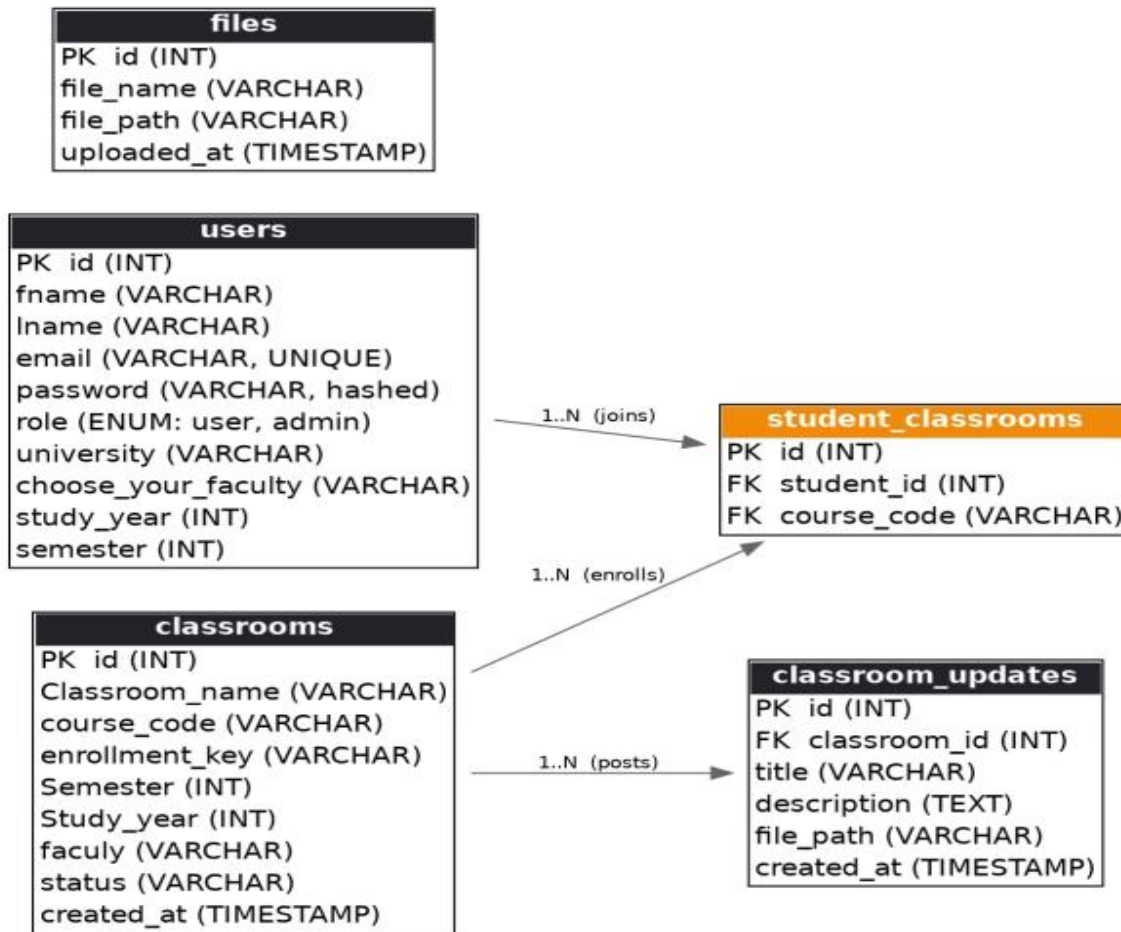


Figure 2: Entity-Relationship diagram of the uniScholar database

Note: the `student_classrooms` table is used in the join logic (`database/student-classroom-join.php`) but its CREATE TABLE statement is missing from `connection.php`. This is flagged and explained in Section 4, along with the fix.

2.3 Folder Structure

The project follows a component-based folder layout so that pages, styles, and database logic are kept separate:

```
uniScholar/
├── auth/                # Login, register, logout, forgot-password pages
│   ├── login.php
│   ├── register.php
│   ├── logout.php
│   └── ForgotPassword.php
├── database/            # PHP scripts that talk to MySQL
│   ├── connection.php  (creates DB + tables if missing)
│   ├── login.php
│   ├── register.php
│   └── admin-classroom-save.php, student-classroom-join.php ...
├── components/          # Reusable page parts and feature pages
│   ├── navbar.php, Footer.php, search.php
│   ├── admin_dashbord.php, student_dashbord.php
│   ├── admin_classroom.php, student_classroom.php
│   ├── student-classroom-view.php (view notices + download files)
│   └── GPA.php, About.php, Contact.php, Course.php ...
├── css/                 # bootstrap.min.css, style.css
├── js/                  # bootstrap.bundle.min.js, gpa_calculator.js
├── img/                 # logos, icons, carousel images, sample PDFs
├── uploads/             # files uploaded by users at runtime
├── index.php            # homepage
└── README.md
```

3. Implementation

This section explains the main features and shows the screens and logic behind them. Because the project needs a live PHP + MySQL server to run, the screenshots below combine the original Figma wireframes (used as the visual design reference) with a real screenshot taken from phpMyAdmin during development, and short code samples from the actual project files. Before final submission, these wireframe images should be swapped for real screenshots taken from the running site.

3.1 User Authentication

New users sign up through `auth/register.php`, which sends data to `database/register.php`. That script checks that all fields are filled, checks the password and confirm-password match, checks the email is not already used, then saves the password using PHP's `password_hash()` function (never as plain text). After signup, the user is sent to a page to complete their university and faculty details.

Login works the same way in reverse: `database/login.php` looks up the account by email, then uses `password_verify()` to check the password against the saved hash. If correct, session variables are set (user id, name, email, faculty, role) and the user is redirected to the admin dashboard or the student dashboard, based on their role.

A

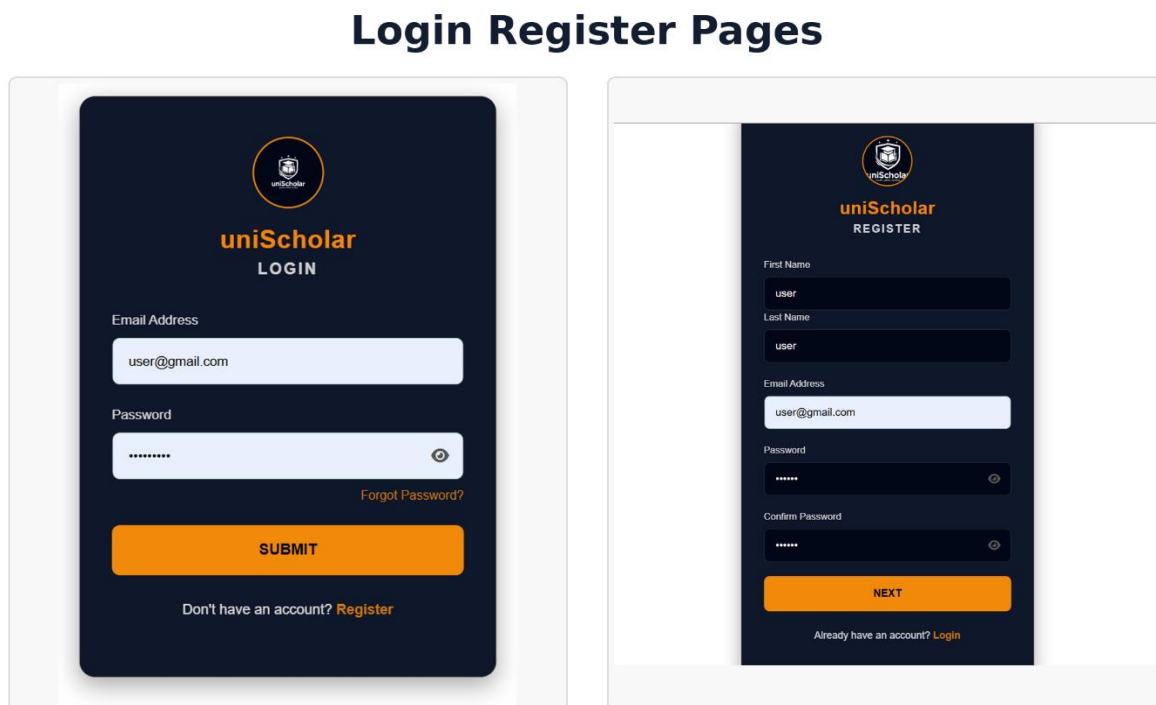


Figure 3: Login and Register screen design (Figma wireframe)

```

/database/login.php (simplified)
if (password_verify($password, $user['password'])) {
    $_SESSION['user_id'] = $user['id'];
    $_SESSION['role'] = $user['role'];
    if ($user['role'] === 'admin') {
        header("Location: ../components/admin_dashbord.php");
    } else {
        header("Location: ../components/student_dashbord.php");
    }
}
}

```

3.2 Role-Based Dashboards

After login, the session role decides which dashboard the user sees. The Admin Dashboard shows total students, total classrooms, total courses, and quick buttons to create a new classroom, course, admission, or broadcast. The Student Dashboard shows the student's department, study year and semester, their joined courses, and shortcuts to Past-Paper, Notes, and the GPA calculator.

Admin Dashboard

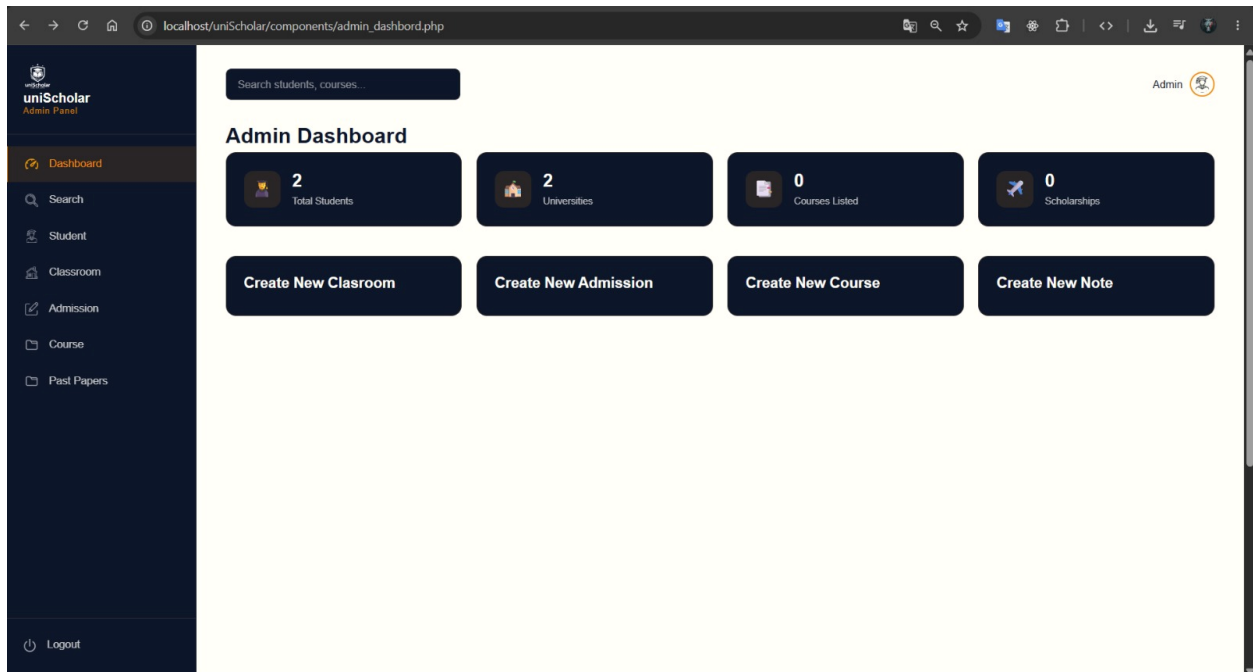


Figure 4: Admin Dashboard design

Student Dashboard

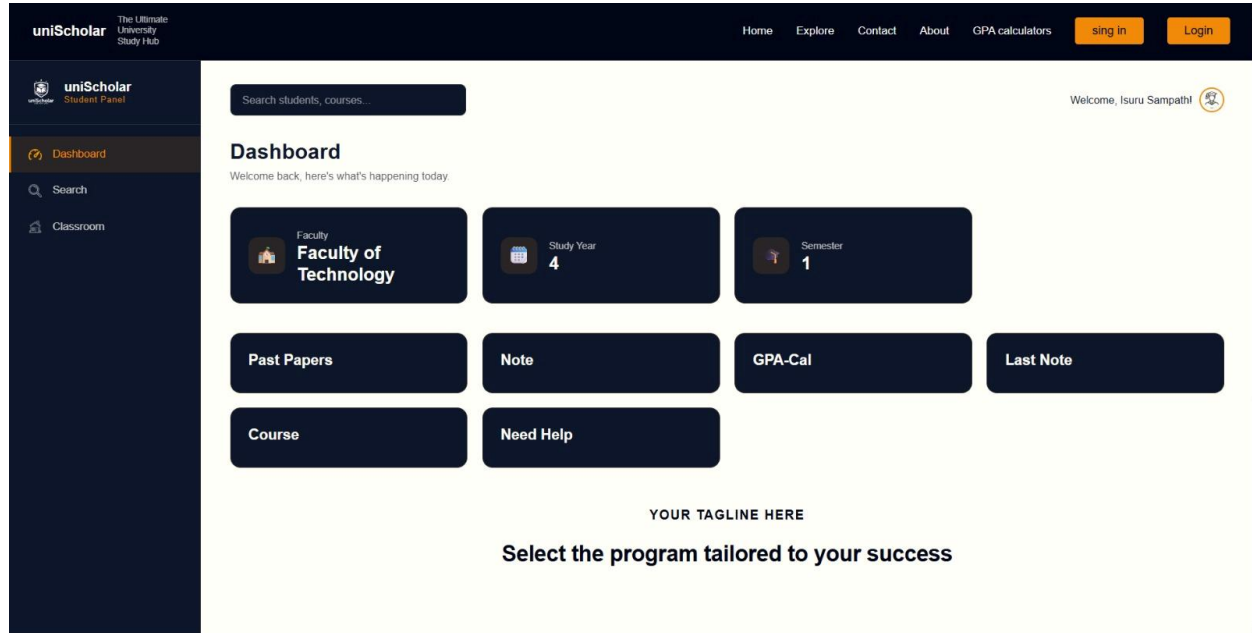


Figure 5: Student Dashboard design

3.3 Classroom Creation and Joining

An admin creates a classroom by filling in a name, course code, semester, study year, faculty, and a private enrollment key. This is saved by `database/admin-classroom-save.php` using a prepared statement, and the code first checks that the same classroom name and course code do not already exist.

A student joins a classroom from the student side by typing the course code and the enrollment key. The script `database/student-classroom-join.php` checks the code and key match a real classroom, checks the student has not already joined it, and then adds a new row to `student_classrooms`.

3.4 Notice Board and File Download

Once a student has joined a classroom, they can open it from their dashboard to see all updates posted by the admin, newest first. Each update shows a title, a description, the date it was posted, and — if a file was attached — a Download Attachment button. This is handled by `components/student-classroom-view.php`, which reads the `classroom_updates` table filtered by `classroom_id`.

```
// components/student-classroom-view.php (simplified)
$updateQuery = $conn->prepare(
    "SELECT * FROM classroom_updates WHERE classroom_id = ? ORDER BY created_at
    DESC"
);
$updateQuery->bind_param("i", $real_classroom_id);
...
if (!empty($row['file_path'])) {
    // show a Download Attachment button linked to the file
}
```

3.5 Live Search

The search box in the navigation bar (components/search.php) uses plain JavaScript. As the student types, the filterSearch() function hides links that do not match and shows the ones that do, updating the dropdown instantly with no page reload. Pressing Enter or clicking Search sends the user to the matching page.

3.6 GPA Calculator (Client-Side JavaScript)

The GPA calculator (js/gpa_calculator.js) lets a student pick a grade for each course, add it to a running list, and remove any entry with one click. Every time a course is added or removed, the script recalculates the GPA instantly in the browser using a grade-point table, with no server call needed.

3.7 Responsive Design and Visual Interest

All pages use Bootstrap 5's grid system so that the layout adjusts automatically between desktop, tablet, and phone screens. The homepage also uses a Bootstrap image carousel (components/image_carousel.php) to show welcome banners, and buttons and cards use smooth CSS transitions (for example, transition: transform 0.2s ease;) so hover effects feel polished rather than instant.

3.8 Database Evidence

The screenshot below, taken from phpMyAdmin during development, confirms that the files table was created successfully with the correct columns, matching the schema described in Section 2.2.

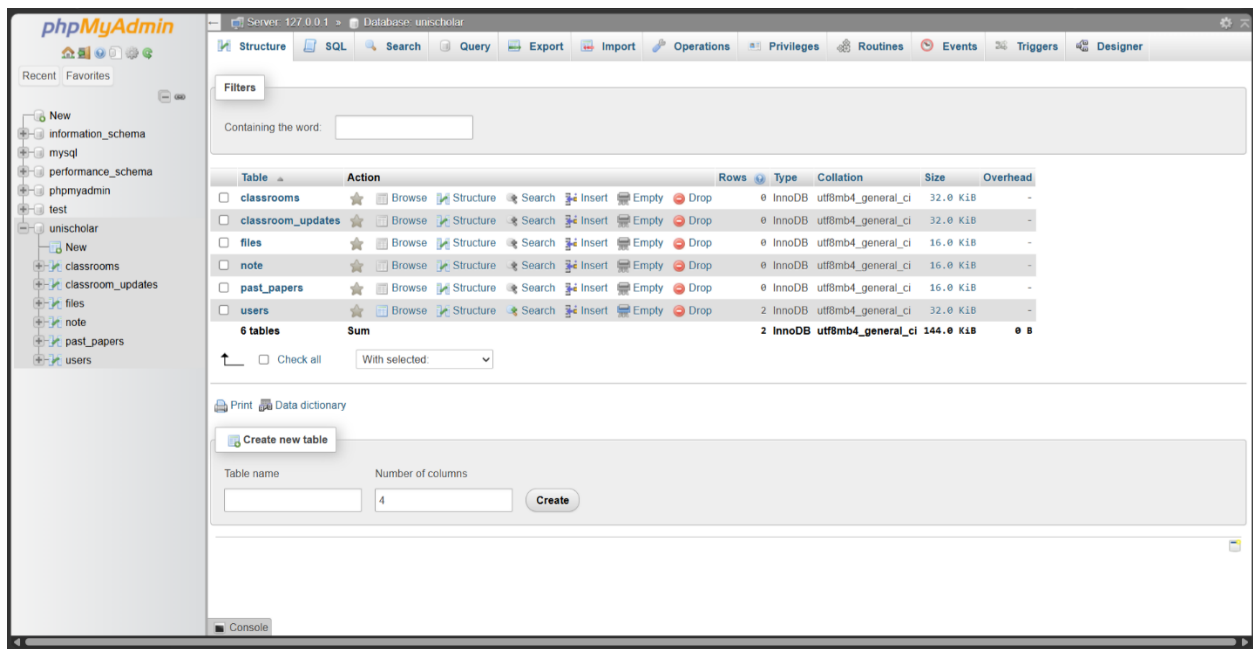


Figure 6: phpMyAdmin view of the files table structure

4. Challenges and Solutions

Building uniScholar came with real problems, just like any web project. Being honest about these problems (and how they were fixed, or how they could be fixed) is part of learning — and it is also useful to bring up in the viva.

Challenge 1: Missing student_classrooms table

The script that lets a student join a classroom (student-classroom-join.php) reads and writes to a table called student_classrooms. But this table is never created in database/connection.php, where all the other tables are set up. This means the join feature would fail on a fresh database with a MySQL error.

- **Solution:** Add a CREATE TABLE IF NOT EXISTS student_classrooms statement in connection.php, with columns id, student_id, course_code, and a UNIQUE key on (student_id, course_code) to stop duplicate joins. This is a quick fix and should be done before final submission.

Challenge 2: Column mismatch when saving a classroom

database/admin-classroom-save.php tries to insert a value into a column called up_date, but the classrooms table (in connection.php) does not have this column. Depending on the MySQL server's strict mode, this either causes an error or silently fails.

- **Solution:** Either remove up_date from the insert statement, or add an up_date column to the classrooms table so both files match. Keeping the database schema and the PHP code in sync is an important lesson from this bug.

Challenge 3: Contact form not connected to the database

The project guide asks for a messages table and a working contact form. In the current code, components/Contact.php only loads the navbar, search box, and footer — there is no form and no messages table anywhere in the project.

- **Solution:** Add a messages table (id, name, email, message, created_at) and a small PHP script that inserts form data using a prepared statement, similar to the pattern already used in register.php and admin-classroom-save.php. Because the same insert pattern is already used elsewhere in the project, this can be completed quickly by copying that pattern.

Challenge 4: Some pages are placeholders

A few planned pages, such as Past-Paper.php and Course.php, are not finished. Past-Paper.php only shows one hard-coded link, and Course.php actually shows a calendar rather than a course page. This suggests time pressure during development, which is normal for a first web project done in pairs.

- **Solution:** These pages should either be completed to match their planned purpose, or removed from the navigation menu until they are ready, so that visitors do not click a link that leads to an unfinished page.

Challenge 5: Learning PHP sessions and prepared statements

Both members were new to combining PHP sessions with MySQL prepared statements. Early commits in the Git history show repeated small fixes to login and registration (for example, commit messages such as "fix registaer erry" and "fix mising (s) data type"), which shows the normal trial-and-error process of learning `bind_param()` type strings (like "ssss" or "iis").

- **Solution:** Reading the official PHP `mysqli` documentation and testing each query directly in phpMyAdmin before wiring it into PHP helped catch type mismatches early.

5. Individual Contribution

The project's GitHub history (135 commits in total) shows steady work from both group members, with each person pushing to feature branches before merging into main. The table below gives a general picture based on the commit history and which files each member's commits mostly touched. Both members should check this table and adjust it if needed, since only the members themselves can confirm the exact split of work for the viva.

Area	Main Contributor(s)	Details
Authentication (login, register, sessions)	A.K. Nisal Nethsara	Shared work on auth/ and database/ login.php, register.php, with several joint bug-fix commits.
Admin dashboard & classroom management	A.M.I.S. Silva	Classroom creation, editing, and the admin dashboard layout.
Student dashboard & classroom join	A.K. Nisal Nethsara	Student dashboard, joining classrooms, and the classroom notice view.
Styling (CSS, Bootstrap layout)	Both members	style.css and Bootstrap component usage were updated by both members across the project.
GPA Calculator (JavaScript)	A.M.I.S. Silva	js/gpa_calculator.js and its dashboard shortcut.
Wireframes & Figma design	Both members	Login/Register, Home, Admin, and Student wireframes.
Version control & documentation	Both members	README.md updates and merge commits on GitHub.

Both members must be ready to explain any part of the code, including parts mainly written by their partner, since the viva tests individual understanding of the whole project, not just each person's own section.

6. Setup Instructions

Follow these steps to run uniScholar on a local computer using XAMPP.

Step 1: Install requirements

- Install XAMPP (or WampServer) with PHP 8 and MySQL.

Step 2: Get the project files

- Clone the repository: `git clone https://github.com/ISURU-SAMPATH-SILVA/uniScholar.git`
- Place the uniScholar folder inside htdocs (for XAMPP: `C:\xampp\htdocs\` on Windows, or `/opt/lampp/htdocs/` on Linux).

Step 3: Start the server

- Open the XAMPP Control Panel and start Apache and MySQL.

Step 4: Set up the database

- Open `http://localhost/uniScholar/database/connection.php` once in a browser. This script automatically creates the unischolar database and its tables (users, classrooms, files, classroom_updates).
- Alternatively, open `http://localhost/phpmyadmin`, create a database named unischolar, and check the tables were created after step above.

Step 5: Check the connection settings

Open `database/connection.php` and confirm the settings match your MySQL server (the default XAMPP settings need no changes):

```
$host = "localhost";  
$dbUser = "root";  
$dbPass = "";  
$dbName = "unischolar";
```

Step 6: Open the application

- Go to `http://localhost/uniScholar/` in your browser.
- Register a new student account, or create an admin account by changing that user's role to 'admin' directly in the users table using phpMyAdmin.

7. References

- PHP Manual – password_hash(): <https://www.php.net/manual/en/function.password-hash.php>
- PHP Manual – MySQLi Prepared Statements: <https://www.php.net/manual/en/mysqli.prepared-statements.php>
- Bootstrap 5 Documentation: <https://getbootstrap.com/docs/5.0/>
- MDN Web Docs – JavaScript Guide: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>
- ICT 1209 Mini Project Guide, Department of ICT, Rajarata University of Sri Lanka (2023/24 Batch).
- uniScholar GitHub Repository: <https://github.com/ISURU-SAMPATH-SILVA/uniScholar>
- uniScholar Figma Design File: <https://www.figma.com/design/jELQAJaf5DkNNVfiscqRnF/uniScholar>